BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS
ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

3D NEURAL CELLULAR AUTOMATA

3D NEURÁLNÍ CELULÁRNÍ AUTOMATY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **MICHAL REPČÍK**
AUTOR PRÁCE

SUPERVISOR **Ing. KAREL FRITZ,**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

This thesis presents a modular 3D neural cellular automata implementation in Python and PyTorch. The project includes the simulation core, a training runner, a network server for remote control, and a Blender add-on for live visualization. The work focuses on reproducibility, voxel I/O, and practical use during training and observation of volumetric growth.

Abstrakt

Tato práce popisuje modulární implementaci 3D neuronových celulárních automatů v Pythonu a PyTorch. Projekt zahrnuje simulační jádro, trénovací runner, síťový server pro vzdálené ovládání a Blender doplněk pro živou vizualizaci. Důraz je kladen na reprodukovatelnost, voxelový vstupně-výstupní řetězec a praktické použití při trénování a pozorování růstu objemových struktur.

Keywords

neural cellular automata, 3D voxel modelling, PyTorch, Blender add-on, generative modelling

Klíčová slova

neuronové celulární automaty, 3D voxelové modelování, PyTorch, Blender doplněk, generativní modelování

Reference

REPČÍK, Michal. *3D Neural Cellular Automata.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Karel Fritz,

3D Neural Cellular Automata

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Karel Fritz I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT and Grammarly to refine the text and correct spelling and other language mistakes. I used Copilot when working on the software.

. . . . . .
Michal Repčík
April 7, 2026

Acknowledgements

I would like to express my sincere thanks to my supervisor, Ing. Karel Fritz, for his guidance and support.

Contents

1 Introduction **3**

2 Abstract **4**

3 Theoretical Foundations **5**
3.1 Prerequisites 5
3.2 Cellular Automata 5
3.3 Neural Cellular Automata 8
3.4 Extension from Two to Three Dimensions 8
3.5 Core Components of a 3D Neural Cellular Automaton 9

4 Related Work **13**
4.1 Growing Neural Cellular Automata 13
4.2 Three-Dimensional Neural Cellular Automata in Interactive Environments . 16
4.3 Open-Source and Applied Three-Dimensional Neural Cellular Automata . . 17
4.4 Chapter Conclusion 18

5 Architectural Design **19**
5.1 Design Goals and Requirements 19
5.2 System Overview 20
5.3 Core NCA Model 21
5.4 Training Pipeline 23
5.5 Server and Networking 24
5.6 Blender Integration 26

6 Implementation **28**
6.1 Core Framework 28
6.2 NCA Model Components 29
6.3 Training Pipeline 33
6.4 Server 34
6.5 Blender Add-on 37

7 Testing **39**

8 Experiments **41**

9 Conclusion **42**

Bibliography **43**

List of Figures

5.1 High-level system architecture. Arrows indicate dependency direction. The dashed TCP boundary marks the network separation between the server and the Blender client . . . 20

5.2 Per-voxel state vector layout. The vector contains hidden channels, optional task channels, and visible channels. The alpha channel, which serves as the alive signal, is always the last visible channel. Task channels are an optional extension and are disabled by default . . . 21

5.3 Data flow through a single NCA update step. The state passes through the fixed 3D perception filters, is transformed by the shared update rule, and is then gated by the alive masking mechanism before producing the updated state . . . 23

5.4 Client-server message sequence for a complete session. The client drives all state transitions; the server responds with acknowledgements and streams state broadcasts during active training or inference . . . 25

5.5 Session state machine. The server transitions between disconnected, idle, training, paused, and inference states in response to client messages. All active states return to disconnected if the client closes the connection . . . 26

6.1 Detailed data flow through a single NCA update step, showing the full layer sequence within the update MLP, the optional stochastic fire mask, and the bounded tanh output . . . 32

6.2 Wire message format. Each message consists of a four-byte big-endian unsigned integer carrying the payload byte length, followed by the UTF-8 JSON payload . . . 36

6.3 Protocol message types and their JSON fields. Messages above the divider are sent by the client; messages below are sent by the server . . . 37

Chapter 1

Introduction

TODO

Chapter 2

Abstract

TODO

Chapter 3

Theoretical Foundations

3.1 Prerequisites

This thesis assumes familiarity with convolutional neural networks, gradient-based optimisation, and automatic differentiation. Goodfellow et al. [7] and the official PyTorch documentation [3] cover these topics in depth.

Everything relating to cellular automata and their neural extensions is introduced from scratch in the sections that follow.

3.2 Cellular Automata

Cellular automata (CA) are discrete dynamical systems composed of a regular grid of cells, each occupying one state from a finite set. All cells update in parallel at each discrete time step according to a local rule that depends only on the states of neighbouring cells [13].

A cellular automaton is defined by:

- a **lattice** of cells, typically arranged in one, two, or three dimensions,
- a **finite state set** $\mathcal{S}$,
- a **neighbourhood function** $\mathcal{N}$ describing which cells influence a given cell,
- an **update rule** $f : \mathcal{S}^{|\mathcal{N}|} \to \mathcal{S}$ that determines the next state [9].

Ulam and von Neumann developed the first formal CA models in the 1940s as a theoretical framework for self-replicating machines [16]. Von Neumann's original construction used a two-dimensional lattice with 29 distinct cell states and demonstrated that a sufficiently complex local rule can produce a system capable of constructing copies of itself without any centralised coordination.

Wolfram later proposed a behavioural classification of CA into four qualitative classes based on their long-term dynamics [27, 28]. Class 1 systems converge to a uniform state regardless of initial conditions. Class 2 systems settle into static or periodic patterns. Class 3 systems produce chaotic, seemingly random output sensitive to initial conditions. Class 4 systems generate complex dynamics with long-lived interacting structures capable of computation. This classification provides a useful reference for characterising the dynamics of both classical and neural CA.

3.2.1 One-Dimensional Cellular Automata

The simplest non-trivial CA operate on a one-dimensional array of cells. In the elementary CA family, each cell reads its own state and its two immediate neighbours (radius 1, neighbourhood size 3). With binary states, $2^3 = 8$ neighbourhood configurations are possible, giving $2^8 = 256$ distinct rules in Wolfram's enumeration [27].

Each rule number encodes a next-state lookup table in binary. For example, Rule 90 assigns the next state of a cell based on the exclusive-or (XOR) of its two neighbours, producing the Sierpiński triangle from a single active seed. This pattern is fractal and self-similar, placing Rule 90 in Wolfram's Class 3 (chaotic behaviour). Rule 30 also belongs to Class 3 and produces irregular, aperiodic evolution that has been used as a source of pseudorandomness [28].

Rule 110 (Class 4) is the smallest known Turing-complete one-dimensional CA. Cook [4] proved that it supports gliders and glider collisions that can simulate any Turing machine, showing that computational universality can arise from a simple local rule on a one-dimensional lattice.

Despite their simplicity, elementary CA exhibit all four classes of Wolfram's classification — a minimal demonstration of how local rules can produce qualitatively different global behaviour. The same question extends directly to the neural CA models discussed later in this chapter.

3.2.2 Two-Dimensional Cellular Automata

In two dimensions, the Moore neighbourhood (3×3 block surrounding each cell) is the most common choice. With binary states and a Moore neighbourhood, the update rule must handle $2^9 = 512$ possible input configurations. Because the full lookup table is large, 2D rules are typically expressed using birth/survival notation. A rule written as Bn/Sm specifies that a dead cell becomes alive (is "born") when it has exactly n live neighbours, and a living cell survives when it has exactly m live neighbours. All other cells die or remain dead.

Conway's Game of Life [6] uses the rule B3/S23: a dead cell becomes alive with exactly three live neighbours, and a living cell survives with two or three live neighbours. Despite the simplicity of this rule, Life produces a wide variety of emergent structures. Still lifes (such as the block and beehive) are stable configurations that do not change between steps. Oscillators (such as the blinker and toad) cycle through a fixed sequence of states. Spaceships (such as the glider and lightweight spaceship) translate across the grid while cycling through their phases. More complex constructions include glider guns, which periodically emit spaceships, and computational structures that perform logic operations.

Berlekamp, Conway, and Guy [1] showed that Life is Turing-complete: any computation that can be expressed as a Turing machine can, in principle, be carried out within a sufficiently large Life grid. This highlights the defining property of cellular automata – complex, globally coordinated behaviour emerging from a homogeneous local rule. The same property later motivated the development of neural cellular automata, where the local rule is learned rather than hand-designed.

3.2.3 Three-Dimensional Cellular Automata

In three dimensions, the Moore neighbourhood covers 26 adjacent cells (6 face-adjacent, 12 edge-adjacent, and 8 corner-adjacent) in a $3 \times 3 \times 3$ cube surrounding the central cell.

Binary states with a full Moore neighbourhood produce $2^{27} \approx 134$ million neighbourhood configurations, making exhaustive rule enumeration impractical. For this reason, practical 3D CA use totalistic or outer-totalistic rules that depend only on the count of live neighbours rather than their exact positions [28].

Several notable 3D CA have been studied. Von Neumann's 29-state self-replicating automaton [16], originally formulated in 2D, has been extended to three dimensions, where the additional spatial freedom allows more compact replicating structures. Excitable-media rules such as Brian's Brain have also been generalised to 3D lattices, producing travelling wavefronts and spiral structures in three-dimensional space [11]. Three-dimensional Game of Life variants using totalistic birth/survival rules generate branching and crystal-like growth patterns that have no direct 2D analogue.

Hardware advances in the 2000s made large-scale 3D CA simulations practical on consumer hardware [25]. GPU-accelerated implementations can now run volumetric CA with grid sizes exceeding 128^3 cells in real time. These volumetric models provided a natural setting for the neural CA extensions discussed later in this chapter, as the underlying lattice structure and local-update paradigm transfer directly from classical to neural formulations.

3.2.4 Differentiability and Continuous Cellular Automata

Classical CA use discrete states and discrete update rules. However, several extensions have shown that the core CA principles (spatial lattice, local interactions, and parallel updates) also work with continuous states and smooth transition functions. These continuous CA are directly relevant to neural CA, which operate on real-valued state vectors and use differentiable update rules trained by gradient descent.

An early theoretical connection was established by Turing [26], whose reaction-diffusion model of morphogenesis describes how spatial patterns can arise from the interaction of locally diffusing chemical substances. Although Turing's model uses partial differential equations rather than a discrete lattice, the underlying principle - local interactions producing global spatial structure - is the same principle that governs cellular automata. This connection between reaction-diffusion systems and CA has motivated much of the subsequent work on continuous and neural CA.

SmoothLife [19] generalises Conway's Game of Life to continuous space and time. It replaces the discrete neighbour count with continuous averaging kernels and the binary birth/survival rule with smooth sigmoid transition functions. The result is a system that runs on a continuous 2D field and produces gliders, oscillators, and wave-like structures that closely resemble their discrete counterparts but evolve smoothly in time. SmoothLife shows that the characteristic behaviours of Life do not depend on discretisation, but arise from the structure of local interactions.

Lenia [2] extends this approach by parameterising both the convolution kernel (which defines the neighbourhood shape and weighting) and the growth function (which determines how the local average maps to a state change). Systematic exploration of Lenia's parameter space has produced hundreds of stable, self-organising patterns that resemble biological microorganisms such as orbiting particles, pulsating organisms, and multi-species ecologies.

Together, SmoothLife and Lenia show that complex self-organisation does not require discrete state transitions – local, differentiable interactions are sufficient to produce stable spatial structures. Neural cellular automata build on this observation: rather than manually exploring the parameter space as in Lenia, neural CA use gradient-based optimisation to

find update rules that produce a specific target pattern. The differentiability of the update rule is what makes this possible.

3.3 Neural Cellular Automata

Mordvintsev et al. [14] introduced Neural Cellular Automata (NCA) as a differentiable generalisation of classical CA. The discrete lookup-table rule is replaced by a small neural network whose parameters are shared across all cells. The spatial lattice stays discrete, but each cell's state becomes a real-valued vector, and the update rule is trained end-to-end by gradient descent.

Let the lattice be $\Omega \subset \mathbb{Z}^d$, where d is the spatial dimension. Each cell $x \in \Omega$ holds a state vector

$$\mathbf{s}_t(x) \in \mathbb{R}^C,$$

where C is the number of channels. The full grid state at time t is a tensor over all cell states.

The update process has two stages [14]. First, a fixed perception operator extracts local neighbourhood information through convolutional filters. The filters are applied to the full multi-channel grid and produce a perception vector at each cell location. Second, a shared neural network maps this perception vector to a state increment $\Delta\mathbf{s}$ that is added to the current state:

$$\mathbf{s}_{t+1}(x) = \mathbf{s}_t(x) + \Delta\mathbf{s}(x).$$

This residual update formulation keeps the defining CA property - strict locality - while replacing the hand-crafted transition function with a learnable model. Because the update rule is differentiable with respect to its parameters, the automaton can be unrolled over multiple time steps and trained by gradient descent to minimise a loss defined on the resulting configuration.

The key conceptual shift compared to classical and continuous CA is that the update rule is not designed or discovered by exploration, but optimised for a specific objective. The system learns to grow a target pattern from a seed, maintain it over time, or regenerate it after damage depending on the training regime. The global behaviour still emerges from distributed local interactions, as in classical CA – but the local rule itself is shaped by optimisation.

3.4 Extension from Two to Three Dimensions

The original NCA of Mordvintsev et al. [14] operates on a 2D grid. Extending it to three-dimensional voxel lattices is conceptually straightforward but introduces several practical differences that affect both the architecture and the training procedure.

In 2D, perception uses 3×3 filters over a nine-cell Moore neighbourhood. In 3D, the same filters generalise to $3 \times 3 \times 3$ volumes covering twenty-seven cells. Prior 3D extensions [24, 21, 10] have shown that Sobel-type gradient filters and other perception operators transfer directly to x, y, and z axes without changing the overall update formulation.

However, volumetric grids introduce two important differences. First, cell count grows cubically with resolution. A 32^3 grid contains 32 768 cells; a 64^3 grid contains 262 144. This cubic scaling increases both memory consumption and computational cost per step, particularly during training when gradients must be stored for backpropagation through multiple steps.

Second, information still propagates only through local interactions, one step at a time. In a 32^3 grid, a signal originating at one corner must traverse roughly 55 steps (the grid diagonal in Manhattan distance through the $3 \times 3 \times 3$ neighbourhood) to reach the opposite corner. In larger grids, this propagation distance grows further. Maintaining coherent global structure therefore requires stable dynamics over longer unrolling horizons than in comparable 2D systems.

As a consequence, behaviours such as persistence and regeneration may behave differently in volumetric domains. The change in neighbourhood structure (27 neighbours instead of 9), spatial topology, and information propagation distance can all affect the stability and character of the emergent dynamics.

3.5 Core Components of a 3D Neural Cellular Automaton

This section describes the principal components of a three-dimensional neural cellular automaton at the conceptual level. The goal is to establish the general design principles that underlie these systems before addressing the specific architecture and implementation in later chapters.

3.5.1 State Representation

Each voxel in a 3D neural CA maintains a vector of C real-valued channels:

$$\mathbf{s} \in \mathbb{R}^C.$$

The channels divide into two functional groups. **Visible channels** represent quantities compared directly to a target pattern, such as colour components (RGB), occupancy, or material properties. **Hidden channels** provide internal memory that is not directly supervised but carries information between neighbouring cells across time steps [14].

The separation between visible and hidden channels is a design choice inherited from the 2D NCA formulation. Visible channels define the observable output of the system, while hidden channels give each cell additional representational capacity to encode information about its local context, history, or role within the emerging structure. Because updates are strictly local, a cell cannot directly observe distant parts of the grid. Hidden channels allow cells to accumulate and relay information over multiple steps, enabling the kind of long-range coordination needed to produce coherent global patterns from local rules.

Most NCA models also designate one channel as a continuous *alive* signal. This channel determines whether a cell is considered active and whether its updates are applied. The alive mechanism appears in the original NCA [14] and has parallels in continuous CA such as SmoothLife [19] and Lenia [2], where real-valued fields govern the activation and deactivation of spatial regions.

3.5.2 Perception Mechanism

Neural CA preserve the locality constraint that defines classical CA: each cell updates based only on its immediate neighbourhood [9, 28]. In a 3D neural CA, locality is realised by applying a fixed set of convolutional filters with kernel size $3 \times 3 \times 3$ to the multi-channel state grid. Each filter reads the 27-cell neighbourhood of every voxel and produces a scalar output per channel [14, 24].

Common filter types include:

- **Identity filters**, which return the cell's own state unchanged. These provide the update rule with information about what the cell currently contains.
- **Neighbour aggregation filters**, which compute a sum or average of adjacent voxels. These capture the local density and composition of the neighbourhood.
- **Gradient filters**, such as Sobel-type kernels extended to the x, y, and z axes. These approximate directional differences and provide a local analogue of gradient sensing in biological morphogenesis [14, 24, 26].

The outputs of all filters applied across all channels are concatenated into a single perception vector per cell. If the state has C channels and G filter groups are used, the perception vector has dimensionality $G{\cdot}C$. This vector is then passed to the update function as input.

The filters are fixed and non-learnable. Using fixed filters constrains the spatial information available to the update rule, which stabilises training and reduces the number of parameters that must be optimised. The approach follows the convention established by Mordvintsev et al. [14] and adopted in subsequent work [24, 21, 10].

3.5.3 Update Rule

The update rule is a shared parametric function that governs how each cell's state evolves over time. It takes the perception vector as input and produces a state increment $\Delta\mathbf{s}$ added to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

This residual formulation encourages incremental modifications rather than abrupt state changes, contributing to dynamical stability over long simulation horizons [14].

The update function is typically implemented as a small multilayer perceptron (MLP) applied independently at each cell. This point-wise application ensures that the update depends only on the locally perceived information, not on any global state. The parameter count is kept deliberately low following the original 2D model, approximately 8 000 trainable parameters [14]. This small scale underlines that the complexity of the resulting structures arises from iterative local interaction, not from network capacity.

Two stabilisation mechanisms appear in most NCA formulations:

- **Alive gating** conditions updates on the alive channel. If a cell and all of its neighbours have alive values below a threshold, the cell's update is set to zero. This prevents empty regions from spontaneously generating structure and provides a soft boundary between active and inactive areas of the grid [14].
- **Stochastic update masking** applies updates to a random subset of cells at each step rather than to all cells simultaneously. This breaks temporal synchronisation artefacts and encourages robustness to asynchronous perturbations [14].

Both mechanisms improve long-term stability and regenerative capacity. They become increasingly important as systems scale to larger grids or higher dimensions, where dynamical instabilities can accumulate over more cells and longer simulation horizons.

3.5.4 Grid Management

The neural CA runs on a finite three-dimensional lattice with fixed dimensions. Boundary conditions (periodic, zero-padded, or reflective) are chosen per experiment and affect the dynamics near the grid edges [14]. Sudhakaran et al. [24] note that the choice of boundary handling varies across implementations and can influence the shapes that the automaton produces.

The primary challenge of 3D grids is their cubic memory scaling. A grid of size N^3 with C channels per cell and 32-bit floating-point precision requires $N^3 \times C \times 4$ bytes of storage per state. For a 32^3 grid with 20 channels, this is approximately 2.5 MB per state. During training, multiple states are held simultaneously (for batching and pool-based sampling), and gradient storage can multiply memory requirements further.

Research on large-scale volumetric simulations addresses this through specialised data structures. Sparse Voxel Octrees [12] exploit the observation that most voxels in a large grid are empty, storing only occupied regions in a hierarchical tree. OpenVDB [15] provides a similar approach with dynamic topology, allowing efficient insertion and deletion of voxels during simulation. While the framework developed in this work uses dense grids at moderate resolution, these sparse representations offer a path toward scaling neural CA to larger volumetric domains in future work.

Initialisation is another practical consideration. Most neural CA start from a *seed* – a single active cell or small cluster placed at the centre of an otherwise empty grid. The alive channel of the seed cell is set to a positive value, and the system must grow the target structure from this minimal starting point through repeated application of the local update rule.

3.5.5 Training and Optimisation

Because the update rule is differentiable, the parameters of a neural CA can be optimised by gradient descent. The standard training procedure unrolls the automaton for T steps starting from a seed state, computes a loss between the resulting configuration and a target, and backpropagates through the full unrolled computation graph [14].

The most common loss function is mean squared error (MSE) between the predicted and target visible channels. For structural objectives, intersection-over-union or other task-specific metrics may be used [17]. The optimiser is typically Adam or one of its variants, with learning rates in the range of 10^{-4} to 10^{-2} [24].

Several training strategies have been developed to encourage specific behaviours:

- **Pool-based sampling** maintains a pool of persistent states at various stages of growth. Each training iteration samples a batch from the pool, runs the automaton forward, computes the loss, and writes the updated states back. This exposes the model to states at different developmental stages rather than always starting from a fresh seed [14].

- **Curriculum learning** gradually increases the number of unrolled steps during training. Early iterations use few steps to learn coarse shape, while later iterations use many steps to refine details and test long-term stability.

- **Training regime selection** determines which aspects of the dynamics are explicitly optimised. These regimes (growth, persistence, regeneration) are discussed in detail in the Related Work chapter.

Backpropagation through many unrolled steps requires storing intermediate activations for the entire sequence, which can exhaust GPU memory. Gradient checkpointing [18] addresses this by discarding intermediate activations during the forward pass and recomputing them during the backward pass, trading approximately twice the computation for substantially reduced memory usage.

Chapter 4

Related Work

This chapter reviews the research most directly related to neural cellular automata and their three-dimensional extensions. The focus is on architectures, training procedures, and experimental results. Each section identifies limitations or open questions that motivate the framework developed in this work.

Several of the works discussed here were already introduced in Chapter 3 to establish formal background. Here, the focus shifts to their concrete design decisions, findings, and the specific choices that inform the approach taken in this work.

4.1 Growing Neural Cellular Automata

Mordvintsev, Randazzo, Niklasson, and Levin [14] introduced the Growing Neural Cellular Automata paradigm that is the methodological basis for this work. They replaced the discrete lookup-table rules of classical CA with a small shared neural network trained end-to-end by gradient descent. The model preserves strict spatial locality while learning to grow and maintain target patterns from simple initial conditions – typically a single live cell.

Unlike earlier continuous CA such as Lenia [2], which rely on manual parameter exploration to discover interesting behaviours, neural CA use supervised optimisation: the transition function is trained to produce a specific target pattern. This work extends that task-driven optimisation approach to three dimensions.

Chapter 3 covered the general NCA principles. The focus here is on the concrete design and empirical properties of the 2020 implementation: its architecture, stabilisation mechanisms, training procedure, and experimental results.

4.1.1 State Representation

Each cell in the 2D model holds a 16-channel real-valued state vector. Four channels encode RGBA colour values compared against a target image. The remaining twelve channels serve as hidden memory.

The hidden channels work as a distributed memory system embedded in the lattice. Because updates are strictly local, long-range coordination must emerge through repeated short-range interactions. A cell cannot observe the state of a distant cell directly; it can only receive information that has been relayed through intermediate cells over multiple steps. The hidden channels provide the capacity for this relay process.

This design is central to the stability and regenerative behaviour observed in later experiments. Reducing the number of hidden channels degrades pattern quality and stability, while increasing it beyond a certain point yields diminishing returns [14]. The balance between enough memory for coordination and a compact state for efficient training directly informs the channel configurations used in subsequent 3D extensions.

4.1.2 Perception Mechanism

Perception uses a fixed set of non-learnable convolutional filters applied to the full multi-channel grid:

- An identity filter, returning each cell's own state,
- Sobel filters approximating spatial derivatives in the x and y directions,
- A Laplacian filter capturing local curvature.

Applied across all 16 channels, these three filter types produce a $3 \times 16 = 48$-dimensional perception vector at each spatial location. The Sobel filters detect directional gradients in the neighbourhood, for example where a pattern transitions from occupied to empty, while the Laplacian identifies edges and boundaries by measuring how much a cell's state differs from its neighbours' average.

A small MLP operating independently at each cell transforms this perception vector into a state increment. The perception step introduces no learnable parameters; all learning occurs in the MLP.

Fixed perception filters are used to ensure locality and translation invariance by design, while also stabilising optimisation by limiting the spatial information available to the update rule. This choice becomes particularly important in volumetric extensions, where the neighbourhood expands from 9 to 27 cells and the perception vector grows correspondingly in dimensionality.

4.1.3 Update Rule and Parameter Sharing

The update rule is a lightweight neural network shared across all cells. It produces a state increment added residually to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

Residual updates encourage incremental modifications rather than abrupt transitions, contributing to dynamical stability. Because the same parameters apply at every spatial location, the learned rule defines a homogeneous dynamical system over the entire grid. No cell has a special role by construction, this means that any differentiation in cell behaviour must emerge from the spatial context encoded in the perception vector.

The full architecture has approximately 8 000 trainable parameters. This small scale is deliberate: it shows that the complexity of the resulting structures comes from iterative local interaction, not from model capacity. By contrast, a standard convolutional neural network for image generation may have millions of parameters. The NCA achieves comparable visual complexity through temporal depth (running many update steps) rather than through network width.

Stabilisation Mechanisms

Two mechanisms are critical for robust long-term dynamics:

- **Alive gating** conditions state updates on whether the designated alpha channel exceeds a threshold in the local neighbourhood. This creates a soft alive/dead boundary and prevents inactive regions from destabilising active structures. Without alive gating, empty regions can spontaneously develop non-zero states due to numerical noise, leading to spurious growth outside the intended pattern.
- **Stochastic update masking** applies updates to a random subset of cells at each step. At each time step, each cell independently decides (with some probability, e.g., 0.5) whether to apply its computed update or skip it. This breaks synchronisation artefacts - patterns that form only because all cells update in lockstep – and produces dynamics that are robust to asynchronous perturbations.

Both mechanisms improve long-term stability and regenerative capacity. Horvath and Levin [8] later demonstrated that stochastic masking also encourages the emergence of distinct cell types within the same NCA, as cells must develop local strategies that work regardless of which neighbours updated in the current step.

4.1.4 Training Regimes

Mordvintsev et al. [14] introduced three training regimes: growing, persistent, and regenerating. Each regime targets a different aspect of the learned dynamics. The choice of regime determines the balance between initial pattern formation, long-term stability, and robustness to damage. These regimes define the training methodology adopted and extended in this work.

Growing Regime

The model is trained to produce a target image from a single active seed. Loss is computed after a fixed number of update steps T, and the model learns to expand outward and approximate the target pattern within that horizon.

No constraint is imposed beyond step T. Patterns may drift, decay, or diverge if simulation continues past the trained step count. In practice, growing-only models commonly exhibit instability within 100–200 steps beyond their training horizon [14].

Persistent Regime

The persistent regime evaluates loss at multiple time steps beyond initial convergence. Rather than stopping at step T, training continues to evaluate the model at steps T, $T+\delta$, $T+2\delta$, and so on, penalising any drift away from the target.

This produces more stable dynamics. Mordvintsev et al. observed that persistent models also exhibit limited regenerative behaviour even without explicit damage training, suggesting that the stability mechanisms learned for persistence partially generalise to perturbation recovery.

Randazzo et al. [20] later extended this idea by training NCA with persistent memory, showing that models trained for stability over very long horizons develop increasingly robust internal representations in their hidden channels.

Regenerating Regime

During training, random damage is applied by zeroing out contiguous rectangular regions of the grid. The model must reconstruct the target from the damaged configuration.

This explicit objective produces the most robust dynamics. Regenerating models recover from substantial perturbations and maintain structural integrity over long simulation horizons. The regenerative capacity is qualitatively reminiscent of biological organisms that can regrow damaged tissue from partial structure [14].

4.1.5 Relevance to This Work

This work adopts the architecture, training regimes, and optimisation strategy of Mordvintsev et al. [14] as its baseline and extends the framework to three-dimensional voxel lattices. The transition from 2D to 3D increases neighbourhood size from 9 to 27 cells, increases computational cost cubically, and extends the information propagation distance required for global coordination. Whether the stability and regenerative behaviours observed in planar domains carry over to volumetric grids is a central question motivating the experiments in this work.

4.2 Three-Dimensional Neural Cellular Automata in Interactive Environments

Sudhakaran et al. [24] extended NCA to three dimensions within the sandbox game *Minecraft*, evaluating volumetric growth of structures and functional machines under real-time constraints and user interaction.

Their model extends the 2D growing NCA paradigm to a 3D voxel grid. Each voxel stores a state vector with visible and hidden channels. Perception uses local $3 \times 3 \times 3$ neighbourhoods with identity and gradient features, processed by a small neural network applied identically at every voxel. The system is trained by gradient descent to grow a target structure from a single seed.

The authors showed that neural CA can produce coherent volumetric structures in a dynamic, interactive environment. The learned rules stay local and homogeneous, preserving the CA paradigm while enabling complex global structure formation. Their experiments showed convergence toward target voxel configurations and partial regenerative behaviour under perturbation: when parts of a Minecraft structure were manually destroyed, the NCA could partially rebuild the damaged region.

An interesting aspect of this work is its evaluation in a constrained real-time environment. The Minecraft platform imposes fixed block types, a specific rendering pipeline, and interaction mechanisms that differ from the unconstrained continuous-valued grids used in purely computational experiments. Despite these constraints, the NCA paradigm produced recognisable structures, showing that the approach is not limited to idealised simulation environments.

4.2.1 Experimental Scope and Constraints

The evaluation focuses on proof-of-concept demonstrations: successful growth toward predefined targets and qualitative assessment of structural coherence. Quantitative metrics for

long-term stability, systematic regeneration under controlled damage patterns, or comparison with other 3D generative methods are limited.

The *Minecraft* environment also constrains the experimental setup. Voxel states are mapped to a discrete set of block types, the rendering pipeline is fixed, and interaction occurs through the game's built-in tools. While this provides a real-time testbed accessible to a broad audience, it limits geometric complexity, material expressiveness, and the precision of experimental measurements available for systematic investigation.

4.2.2 Relation to This Work

Sudhakaran et al. [24] provide the methodological basis for extending neural CA from planar grids to volumetric $3 \times 3 \times 3$ perception neighbourhoods. Strict locality, parameter sharing, and gradient-based optimisation form the conceptual foundation adopted here.

The framework developed in this work embeds the 3D neural CA within Blender instead of a game engine. Blender provides programmatic access to volumetric grids, mesh representations, rendering pipelines, and Python scripting through its built-in API. This gives precise control over lattice resolution, boundary conditions, material properties, and intermediate-state inspection.

The goal is not only to demonstrate 3D feasibility, but to provide a research-oriented framework for controlled investigation of stability, persistence, and regeneration in volumetric neural CA. Where the Minecraft work showed that 3D NCA can produce visually convincing results in a game environment, this work aims to provide the tools for systematic experimentation and analysis.

4.3 Open-Source and Applied Three-Dimensional Neural Cellular Automata

Beyond the work of Mordvintsev et al. [14] and the interactive extension by Sudhakaran et al. [24], several open-source projects have explored three-dimensional neural CA. This section reviews implementations whose design decisions are directly relevant to the framework developed in this thesis.

4.3.1 Standard 3D Reconstruction Baseline

The repository of Aadityaza [10] implements a direct extension of the Growing NCA paradigm to three-dimensional space. It uses a Jupyter Notebook environment and PyTorch to reconstruct voxel shapes from a single seed.

This project provides a clean verification of 3D perception operators, specifically the expansion of Sobel-type filters into tri-axial gradient kernels for the x, y, and z axes. The implementation confirms that the convolution-based perception approach used in 2D NCA applies directly to volumetric tensors without losing spatial locality. For this work, the project served as an early reference point for validating the feasibility of the 3D extension.

The implementation is focused on single-target reconstruction and does not include a training server, interactive visualisation, or modular architecture for experimenting with different configurations. These are gaps that the framework developed in this work addresses.

4.3.2 Voxel-Based Morphogenetic Experiments

Rossi and collaborators [21] explored voxel-based neural CA for three-dimensional morphogenesis. Their implementation extends the 2D growing NCA to volumetric grids with $3 \times 3 \times 3$ perception filters and hidden channel representations.

The system trains to approximate 3D voxel targets from simple seeds. Its architecture follows the 2D formulation closely: fixed perception filters, residual updates, shared parameters, and gradient-based optimisation. The experiments confirm that volumetric growth works under strict locality constraints. However, the evaluation is limited to demonstrating successful growth toward targets. Systematic analysis of long-term persistence, regeneration capacity, or scaling behaviour under increasing resolution is not provided.

Both open-source projects provide proof-of-concept validation for 3D morphogenetic NCA but leave open questions about stability and robustness in larger volumetric domains. They also lack integration with a 3D modelling environment that would allow researchers to interactively visualise and manipulate the NCA during training.

4.4 Chapter Conclusion

The works reviewed in this chapter established the architectural principles – local perception, shared neural update rules, residual dynamics, and gradient-based optimisation – that form the methodological basis for this work. The original 2D Growing NCA demonstrated that a small neural network with fewer than 10 000 parameters can learn to grow, maintain, and regenerate complex patterns through purely local interactions. Subsequent three-dimensional extensions confirmed that these principles transfer to volumetric grids and interactive environments, though systematic evaluation in 3D remains limited.

The framework developed in this work builds on these foundations. It integrates volumetric perception, shared update networks, and structured training regimes within a controlled 3D setting. Unlike prior implementations, it is designed to be modular and extensible, providing a general experimental environment where different architectures, training strategies, and evaluation procedures can be compared systematically. The integration with Blender provides interactive visualisation and mesh-based target specification, addressing the tooling limitations identified in existing open-source 3D NCA projects.

Chapter 5

Architectural Design

This chapter presents the design of the 3D NCA framework and its overall architecture. It introduces the key components that form the framework and describes how these components interact with each other. The aim is to provide a clear conceptual overview of the system and explain the roles of its individual parts. To maintain focus on design, implementation-specific details such as the libraries and frameworks chosen are intentionally omitted here and discussed separately in the Implementation chapter.

5.1 Design Goals and Requirements

The purpose of the framework is to provide an environment for experimenting with neural cellular automata operating on three-dimensional voxel grids. To support experimentation and future extensions, the architecture requires a clear structure and well-defined interactions between its components.

The primary design goal is to enable simulation and training of 3D neural cellular automata while maintaining a modular architecture. The system should allow individual components, such as the simulation grid, update mechanism, and training pipeline, to be developed and modified independently. This simplifies experimentation with different models and update rules.

A second objective was to keep the core functionality independent from visualisation tools and external applications. The framework is therefore designed as a standalone module that provides the core functionality required for initialising and training neural cellular automata. Other systems can build on this module and reuse its computational components.

During development, practical constraints related to visualisation and execution environments influenced the design. Combining interactive visualisation with computationally intensive training within a single environment proved difficult. These limitations motivated a separation between computation and visualisation.

As a result, the system evolved toward an architecture where the core framework provides the underlying simulation and training functionality, while additional components use this functionality to provide visualisation and interaction. This approach preserves the independence of the framework while allowing it to be integrated into different workflows.

5.2 System Overview

The framework is organised into three layers that work together to provide an interactive environment for training and visualising three-dimensional neural cellular automata. Figure 5.1 shows the high-level structure and the communication paths between these layers.

The innermost layer is the **core framework**. It contains the NCA model and the training pipeline and has no dependency on any server or graphical environment. It can be used directly from Python scripts or Jupyter notebooks, as the accompanying experiment notebooks demonstrate.

The second layer is the **server**, which exposes the core framework over a TCP connection. A running server accepts a client connection, receives training configuration and target data, delegates execution to the core training pipeline, and streams state snapshots after each epoch back to the client as training progresses.

The third layer is the **Blender add-on**, which acts as graphical interface and a client. It connects to the server over TCP, provides a panel-based interface for configuation and control over training sessions, and renders incoming state broadcasts as three-dimensional voxel meshes in the Blender viewport.

This separation serves two purposes. Training can run on a remote GPU machine or a Jupyter notebook while the user interacts with Blender on a local workstation, with a tunnelling service bridging the two. At the same time, the core framework remains independent of both the server and the graphical environment, so researchers who do not need interactive visualisation or want to use their own training algorithms can import and use it directly in their own scripts or notebooks.

Alongside the main client–server workflow, the repository also contains lightweight notebook utilities for basic mesh-to-voxel conversion and offline visualisation. These helpers were used during early exploration and intentionally cover only simple targets. They do not preserve Blender materials and do not provide per-voxel colour sampling. The Blender add-on replaces them for practical target preparation and interactive visualisation.

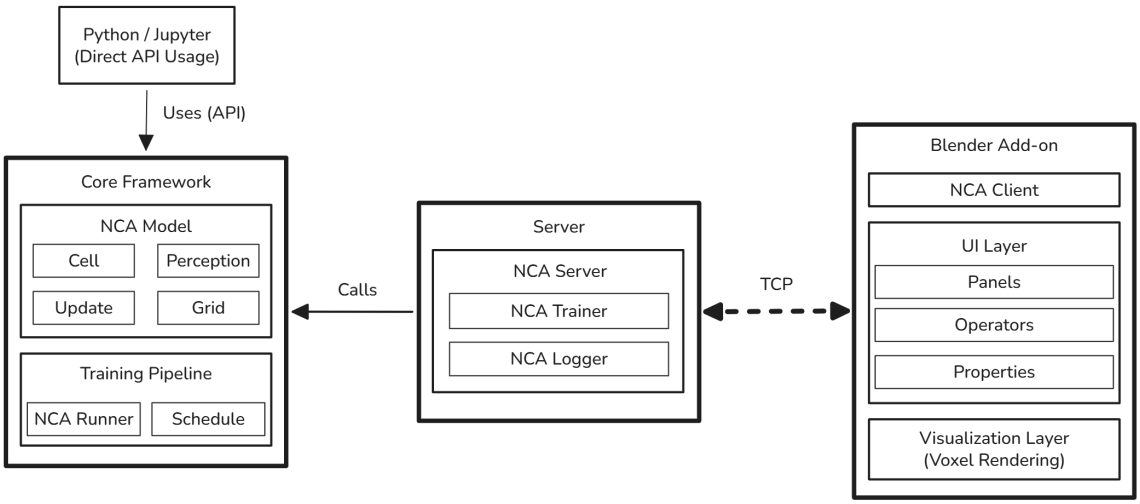

Figure 5.1: High-level system architecture. Arrows indicate dependency direction. The dashed TCP boundary marks the network separation between the server and the Blender client

5.3 Core NCA Model

The core NCA model is composed of four components: `CellState`, `Perception3D`, `UpdateRule`, and `Grid3D`. Each component has its own configuration and can be instantiated independently. `Grid3D` acts as the composition root, wiring the other three together and exposing a single-step forward pass. A higher-level wrapper, `NCAModel`, bundles all configurations into one object and delegates to `Grid3D` internally, providing a simpler entry point for common use cases.

Keeping configurations and components separate means that a researcher who wants to experiment with a custom update rule or a different perception kernel can replace just that component while leaving the rest of the pipeline unchanged. The experiment notebooks included with this thesis illustrate this directly: they use the model components without the runner or the server.

5.3.1 Cell

The cell component defines the structure of the per-voxel state vector and is responsible for computing the alive mask that gates cell updates. It is configured by specifying the number of hidden channels, the number of visible channels, and the alive threshold.

The state vector of each voxel contains three groups of channels, as shown in Figure 5.2. Hidden channels carry internal memory that is not directly supervised and is not compared against the target during training. Visible channels represent the quantities compared against the target, such as RGBA colour values or a single occupancy channel. An optional third group, task channels, allows a single model to be trained on multiple targets simultaneously using a one-hot encoding to distinguish between them. Task channels remain constant for the duration of a training phase, while hidden and visible channels evolve freely under the learned update rule.

Task channels are optional and are disabled in the baseline configuration. They were introduced later as an extension for an experiment that demonstrates modularity and extensibility; single-target training uses only hidden and visible channels.

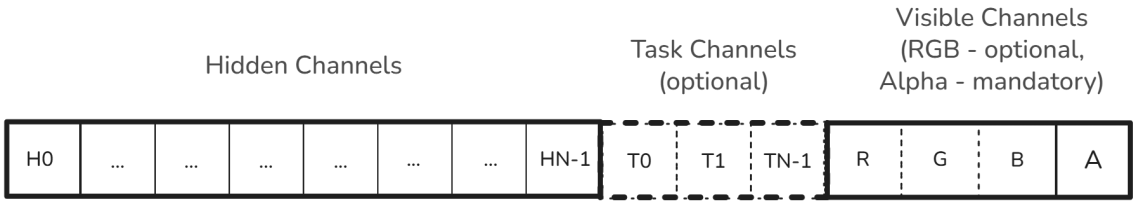

Figure 5.2: Per-voxel state vector layout. The vector contains hidden channels, optional task channels, and visible channels. The alpha channel, which serves as the alive signal, is always the last visible channel. Task channels are an optional extension and are disabled by default

The alive mask controls which regions of the grid are active. It is computed by pooling the alive channel over the local neighbourhood of each voxel and comparing the result against the configured threshold. A cell is considered potentially alive if either it or at least one of its immediate neighbours carries a value above the threshold. This neighbourhood pooling has an important consequence for growth: a dead cell adjacent to an alive cell is

included by the mask, which allows it to receive nonzero updates and transition to an active state over time [14].

5.3.2 Perception

The perception component converts the current grid state into a per-cell representation by applying a fixed set of three-dimensional convolutional filters over the local neighbourhood.

The filters are non-learnable; their weights are fixed at initialisation and do not change during training. Three filter groups are applied across all channels. The identity filter returns each cell's own current state unchanged, giving the update rule direct access to what the cell already contains. The neighbour sum filter aggregates the six face-adjacent neighbours, capturing the local occupancy and composition of the immediate neighbourhood. The Laplacian filter measures how much a cell's state differs from its neighbourhood average, encoding local spatial variation. Applying all three groups across all C channels produces a perception vector at each cell location with dimensionality $3 \times C$.

Keeping the filters fixed follows the original design of Mordvintsev et al. [14]. In practical terms, this means the perception stage has no trainable weights and cannot overfit on its own. Translation invariance is built in by construction through the filter layout, not learned implicitly from data. The only trainable part of this stage is therefore the update rule.

5.3.3 Update Rule

The update rule maps each cell's perception vector to a state increment. Its configuration controls the capacity of the underlying network and whether stochastic update masking is active.

As in Mordvintsev et al. [14], the increment is added residually instead of replacing the previous state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

This tends to produce gradual local adjustments rather than abrupt jumps, which is important when the model is unrolled for many simulation steps.

The same update network is applied independently at every cell. This preserves the locality constraint, because each increment depends only on local perception. It also keeps the dynamics spatially homogeneous: no cell has a privileged role by design. Any functional differences between cells must emerge from their local context in the grid.

When stochastic masking is enabled, each step updates only a random subset of cells. Stochastic masking avoids lockstep synchronisation artefacts and produces dynamics that tolerate asynchronous perturbations, which proves essential for long-horizon persistence and regeneration experiments [14].

5.3.4 Grid and Model Composition

`Grid3D` composes the three components above into a module that executes one complete NCA update step. Figure 5.3 shows the data flow through a single step.

A single step proceeds as follows. The current state is passed to the perception component, which applies the fixed filters and produces a perception vector at every cell location. The update network produces a state increment, which is added residually to the current state. A post-step alive mask is then computed from the updated state, and any cell that falls below the alive threshold has its state zeroed out.

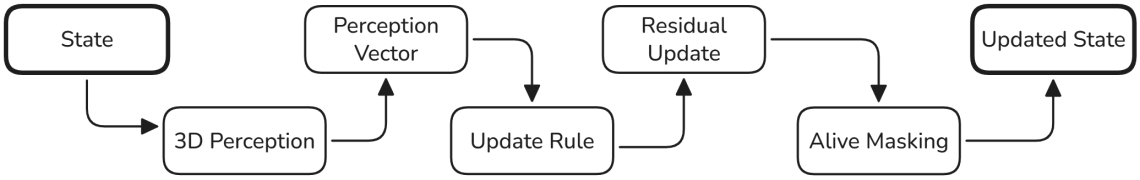

Figure 5.3: Data flow through a single NCA update step. The state passes through the fixed 3D perception filters, is transformed by the shared update rule, and is then gated by the alive masking mechanism before producing the updated state

At the architectural level, masking keeps activity close to the part of the structure that is developing and helps avoid random activation in empty regions. As a result, growth stays more controlled and the dynamics remain more stable over longer runs.

`NCAModel` is a higher-level entry point that bundles all component configurations and delegates to `Grid3D` internally. It additionally handles model persistence: a saved checkpoint stores the configuration alongside the learned weights, making the checkpoint fully self-describing and loadable for inference without access to the original training setup.

5.4 Training Pipeline

The training pipeline sits above the core NCA model and provides the infrastructure needed to optimise the model parameters toward a target voxel configuration. It consists of two components: a training runner, which implements the training loop, and a schedule manager, which handles dynamic parameter changes during training. In the current implementation, the runner boundary is defined by an abstract `NCARunner` interface. The default implementation is `MorphRunner`, but the server can inject a custom runner backend without changing protocol or Blender code.

Alongside this baseline, the core package now includes a regeneration-focused implementation, `RegenRunner`. It applies random local damage to pooled states during training so recovery behaviour can be learned through the same server and protocol path. More channel-centric variants (for example metamorphogenesis with explicit task-channel switching) are kept as notebook-level runners rather than fixed core classes.

5.4.1 Runner Interface and Default Implementation

The default `MorphRunner` implements pool-based training. Rather than restarting from a fresh seed on every iteration, it maintains a fixed-size pool of persistent states representing the grid at various stages of development. Each training iteration samples a batch from the pool, runs it forward for a randomly chosen number of steps, evaluates the result against the target, and writes the updated states back. The sample with the highest loss is replaced with a fresh seed, maintaining pool diversity and preventing convergence to a single developmental stage [14].

From the server perspective, any backend that implements `NCARunner.init()`, `train()`, `snapshot()`, `set_target()`, and `on_event()` can be used as a drop-in replacement.

For experimental work, this interface supports a notebook-first workflow: a custom runner can keep additional experiment parameters hardcoded in notebook code and treat `init(config, target)` primarily as a server handshake. This is practical for non-standard

loops, where changing notebook code is often faster than repeatedly extending Blender UI properties.

In this setup, sending a minimal or dummy `config/target` pair from Blender can still be valid for visualisation-only sessions, provided the custom runner intentionally ignores these values and emits proper snapshots. The key requirement is that `snapshot().state` remains in the expected internal format `[B, C, D, H, W]` and reports a consistent `visible_channels` value so server broadcasting and Blender rendering remain compatible.

The training objective has three weighted parts. Alpha loss compares the predicted and target alive channels. Colour loss compares RGB values only in areas where the target is present, so shape and colour are learned together. Overflow loss penalises activity outside the target, which helps limit unwanted growth. All three weights are configurable and can be changed at runtime through the schedule system.

The number of forward steps applied per training iteration increases over the course of training according to a linear curriculum. Early iterations use short unrolls, which keep gradient paths manageable and allow the model to learn coarse shape quickly. Later iterations use longer unrolls, which test whether the learned rule maintains the target configuration over extended simulation horizons. Both the minimum and maximum step counts increase linearly over a fixed number of epochs and then stabilise at their final values.

5.4.2 Schedule

The schedule component manages one-shot events that fire at specified training epochs. Supported events include learning rate changes, batch size adjustments, individual loss weight updates, and target replacements (changing the objective mid-session) in the default runner implementation. Events are thread-safe: the training loop and server message handler access them via a lock. A special epoch value (NOW = -1) fires an event immediately rather than waiting, allowing users to adjust parameters during live training without restarting.

At execution time, the schedule forwards each due event to the runner via `on_event(event)`. The event is one-shot and removed from the queue after the attempt; if a custom runner does not handle a given event type, the event is reported as not handled.

5.5 Server and Networking

The server layer exposes the training pipeline over a TCP connection. It consists of three main components: a connection manager, which handles clients and dispatches messages; a session runner, which executes training and inference; and a logger, which persists run metadata and artefacts to disk. A shared protocol module defines the message format used across the boundary between the server and the Blender client.

5.5.1 Connection Architecture

The server binds to a configurable host and port and accepts one client at a time. For each connection, it enters a message dispatch loop: route incoming messages to the appropriate session operation and send responses back. When the client disconnects or sends stop, any active session is terminated and the server returns to listening. The design separates command handling from continuous state delivery, following the same general idea as RTSP/RTP systems, where session control and streamed data have different roles [23, 22].

The session runner executes training or inference independently of socket I/O. This keeps protocol handling responsive and maintains clear session lifecycle boundaries.

Figure 5.4 shows the full sequence of messages exchanged during a typical session, from connection through training, schedule update, pause and resume, to inference and final disconnect.

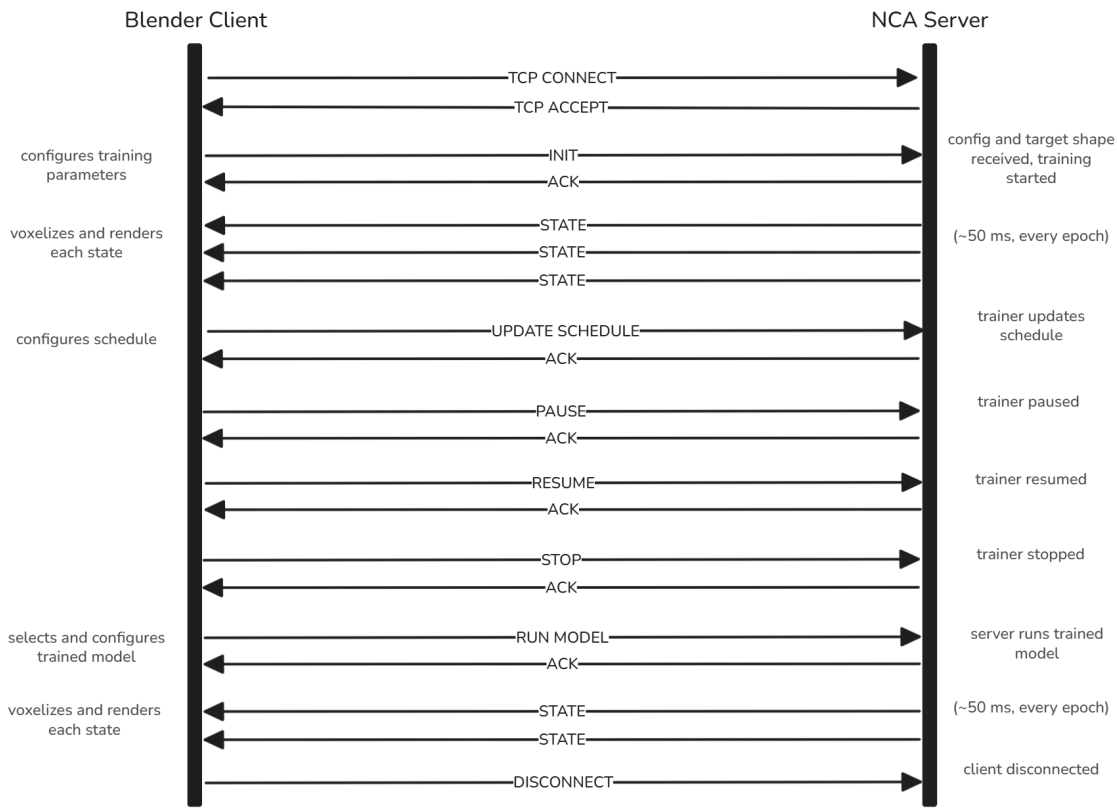

Figure 5.4: Client-server message sequence for a complete session. The client drives all state transitions; the server responds with acknowledgements and streams state broadcasts during active training or inference

Figure 5.5 shows the session state machine from the server perspective. A session begins when the client sends an `init` message for training or a `run_model` message for inference, and ends when the client sends `stop` or disconnects. Training sessions additionally support `pause` and `resume` transitions. Inference does not support pause: once started, it runs until stopped or until the configured number of forward steps is complete.

5.5.2 Protocol

Transport uses TCP, which provides an ordered, reliable byte stream but does not preserve message boundaries [5]. For this reason, the protocol uses explicit framing: each message consists of a fixed-size length header followed by a UTF-8 JSON payload. Binary artefacts (for example state snapshots and checkpoints) are carried as encoded byte strings inside the JSON object, keeping a single self-describing message format that remains readable during debugging.

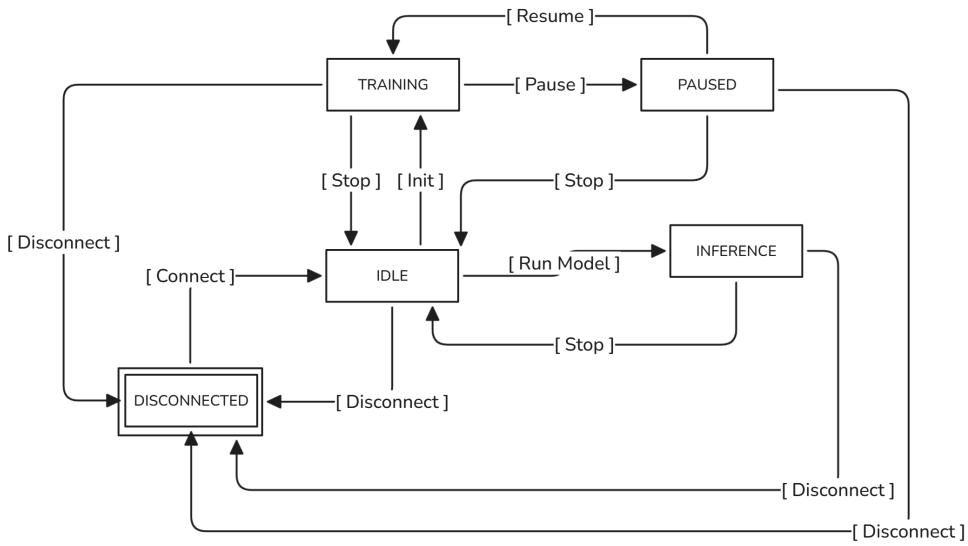

Figure 5.5: Session state machine. The server transitions between disconnected, idle, training, paused, and inference states in response to client messages. All active states return to disconnected if the client closes the connection

The protocol supports two communication directions. Clients send control messages: `init` (start training), `run_model` (start inference), `stop`, `pause`, `resume`, and `update_schedule`. Servers respond with `ack` confirmations or `error`. During training, the server streams `state` messages containing the current grid snapshot, epoch number, and loss value. The client drives all state transitions; the server only streams state and responds to requests.

Although the message vocabulary is domain-specific, the interaction pattern follows established streaming-control designs: client-driven session transitions, periodic state delivery while active, and explicit acknowledgement or error responses to control requests [23, 22].

5.5.3 Logging

The logger saves training metadata and results to a structured directory. Each training run occupies its own directory identified by an automatically assigned run ID, so concurrent or sequential runs never collide. The stored artefacts support reproducibility, analysis, and reuse. Metadata captures full run configuration and timing information, loss files and schedule-event logs record training progress, and checkpoints store serialised models for later use.

Each checkpoint stores the model configuration alongside the learned weights, making it self-describing: it can be loaded and used for inference without access to the original training configuration. The checkpoint interval is configurable, and a final checkpoint is always saved when training completes regardless of the interval setting.

5.6 Blender Integration

The Blender add-on is the primary graphical interface for the framework. It is responsible for three concerns: voxelization of mesh targets, user-facing configuration and control panels, and viewport visualisation of training progress.

5.6.1 Voxelization

The add-on converts Blender mesh objects into voxel arrays that serve as training targets. A mesh selected in the viewport is sampled onto a three-dimensional grid whose dimensions are set by the user. The voxelized result is displayed in the viewport alongside the original mesh so the user can inspect the discretization before starting training.

Material colour information is preserved during voxelization. Each occupied voxel is assigned the base colour of the material covering the nearest surface point, so a voxelized target retains the colour properties of the source mesh.

Voxelized targets can be exported to disk for use in external training scripts or notebooks. This allows users who cannot or prefer not to use the Blender add-on to still benefit from its voxelization capabilities.

5.6.2 User Interface

The add-on provides a hierarchical panel layout in the Blender sidebar, organised under a single root panel. The panels cover connection settings, training control, target configuration, inference parameters, individual model component settings, and schedule management.

Connection state determines which controls are active. Configuration panels are disabled during an active session so that the user cannot modify settings that cannot be changed mid-training. Starting training requires a voxelized target and a connected server. Starting inference requires a connected server and a path to a saved model checkpoint. The two modes are mutually exclusive and both return the server to an idle state when stopped.

5.6.3 Visualisation

State broadcasts received from the server are rendered as voxel meshes in the Blender viewport. The add-on maintains three spatial slots arranged side by side along one axis: the original source mesh, the voxelized target, and the current NCA state received from the server. Placing all three in the same scene lets the user compare the source, the discretized target, and the evolving model output simultaneously.

The state mesh is rebuilt from each received broadcast. A voxel cap prevents rendering from overwhelming the GPU when the grid contains an unusually large number of active cells; in that case only the highest-alpha voxels are retained for display.

Before rendering, a dedicated conversion step maps received state tensors to the display voxel format used by Blender. Treating this as an explicit adapter keeps visualisation agnostic to internal experiment details and makes it straightforward to support alternative state layouts by changing conversion logic rather than rewriting protocol framing.

The network listener and the Blender main thread, which owns all scene data, run separately. The listener receives incoming states and passes them to the main thread, which handles all viewport updates. This keeps the listener responsive and ensures that scene data is only ever modified from the correct thread.

Chapter 6

Implementation

This chapter describes how the framework is implemented in code. It summarises the project structure, the main libraries, and the NCA model components shared by training scripts, the TCP server, and the Blender add-on.

6.1 Core Framework

The implementation is organised into a small number of Python packages:

- `src/core/`: NCA model, stepping logic, and training utilities,
- `src/server/`: TCP server, protocol handling, and the training runner,
- `src/io/`: legacy import/export and mesh conversion utilities used mainly in early notebooks,
- `src/viz/`: notebook visualisation helpers for offline inspection,
- `blender/`: Blender add-on for voxelisation, UI, and viewport rendering.

The notebook utilities are not part of the interactive Blender workflow. Voxel targets for training are prepared and exported through the add-on, which samples colour from Blender materials and provides a single place to adjust grid size and training presets.

PyTorch is used for tensor operations, 3D convolutions, and automatic differentiation required for unrolling the automaton through time [18, 3]. NumPy is used at I/O boundaries and for serialisation-friendly data structures.

The simulator stores the full grid state as a single float32 tensor `state` of shape `[B, C, D, H, W]`. This matches the input convention of `torch.nn.Conv3d` and avoids unnecessary transposes inside the perception and update steps. Listing 6.1 shows the two slices used most often.

```
# state: [B, C, D, H, W]
# V: number of visible channels (e.g. 4 for RGBA)
visible = state[:, -V:, ...]
alpha = state[:, -1:, ...]
```

Listing 6.1: State tensor layout and common channel slices used across the framework.

Channel ordering follows the state vector layout shown in Figure 5.2. Hidden channels come first, optional task channels come next, and visible channels come last, with alpha stored in the final channel.

Task channels are disabled in the baseline configuration and were introduced later as an optional extension used in a modularity experiment.

Model hyperparameters are represented by small configuration dataclasses. Component-level parameters are stored in frozen dataclasses (`CellConfig`, `PerceptionConfig`, `UpdateConfig`, `GridConfig`) so they can be passed around safely and serialised for logs. `NCAConfig` collects the most common parameters into a single object used by the high-level `NCAModel` wrapper.

The interactive client-server setup exchanges configuration and voxel targets in serialisation-friendly formats. The concrete dictionary structure and the conversion between external voxel arrays and internal tensors are described in Sections 6.3 and 6.4.

6.2 NCA Model Components

The architectural roles of the NCA components are described in Section 5.3. This section focuses on how that design maps to Python modules, configuration objects, and entry points.

The NCA model follows the standard two-stage structure: a fixed perception operator extracts local features and a learned update rule predicts a per-voxel state increment. In code, the implementation is split into four modules:

`CellState` channel layout and alive-mask computation,

`Perception3D` fixed depthwise `Conv3d` for local perception,

`UpdateRule` point-wise MLP implemented with $1 \times 1 \times 1$ convolutions,

`Grid3D` one-step update logic and multi-step unrolling.

A thin wrapper `NCAModel` provides a single entry point and standardises checkpointing.

Two construction styles are used in the codebase. The server and runner build `Grid3D` directly from component configs, while notebooks and scripts can use `NCAModel` either with a single `NCAConfig` or directly through keyword arguments that mirror the same fields. Listing 6.2 shows both patterns.

```
# Low-level construction (server/runner)
cell_cfg = CellConfig(hidden_channels=16, visible_channels=4, alive_threshold=0.1,
     task_channels=0)
perc_cfg = PerceptionConfig(kernel_radius=1, channel_groups=3)
upd_cfg = UpdateConfig(hidden_dim=128, stochastic_update=False, fire_rate=0.5)
grid_cfg = GridConfig(size=(32, 32, 32))
grid = Grid3D(cell_cfg, perc_cfg, upd_cfg, grid_cfg)

# High-level wrapper (scripts/notebooks)
model = NCAModel(grid_size=(32, 32, 32), hidden_channels=16, visible_channels=4,
    alive_threshold=0.1)
```

Listing 6.2: Two entry points for constructing the NCA model: low-level `Grid3D` and the high-level `NCAModel` wrapper.

6.2.1 CellState

The per-voxel state vector layout (hidden, optional task, visible) follows Figure 5.2. In code, the layout is specified by `CellConfig` and `CellState.update_alive_mask()` computes the boolean alive mask used to gate updates. A voxel is considered alive if the maximum alpha value in its $3 \times 3 \times 3$ neighbourhood exceeds `alive_threshold`. Listing 6.3 shows the core computation.

```
alpha = state[:, -1:, ...]
pooled = F.max_pool3d(alpha, kernel_size=3, stride=1, padding=1)
alive = pooled > alive_threshold
```

Listing 6.3: Alive-mask computation from the alpha channel.

6.2.2 Perception3D

`Perception3D` implements the fixed perception stage as a depthwise `Conv3d`. With three filter groups per input channel, the perception tensor has `3C` channels. The kernels correspond to identity, face-neighbour aggregation, and a discrete Laplacian. Weights are initialised once and kept fixed during training.

Listing 6.4 shows the kernel patterns in a compact, conceptual form.

```
# identity
K_id[1, 1, 1] = 1

# 6-neighbour sum (face-adjacent voxels)
for (dz, dy, dx) in [(1,0,0), (-1,0,0), (0,1,0), (0,-1,0), (0,0,1), (0,0,-1)]:
    K_sum[1+dz, 1+dy, 1+dx] = 1

# discrete Laplacian
K_lap[1, 1, 1] = 6
K_lap[K_sum == 1] = -1
```

Listing 6.4: Perception kernels used for each input channel (conceptual form).

6.2.3 UpdateRule

`UpdateRule` maps perception features to a per-voxel increment `dx`. It is implemented as a point-wise MLP using $1 \times 1 \times 1$ convolutions, so spatial mixing is handled entirely by perception. The final layer is initialised to zero so the initial dynamics stay close to an identity update.

The raw output is bounded as `tanh(dx)` $\times$ `0.1` to keep individual updates small when the automaton is unrolled for many steps. An optional stochastic fire mask applies the update only to a random subset of voxels per step.

Listing 6.5 shows the layer sequence used in the implementation.

```
mlp = nn.Sequential(
    nn.Conv3d(3*C, H, 1),
    nn.GroupNorm(4, H),
    nn.ReLU(inplace=True),
    nn.Conv3d(H, H, 1),
    nn.GroupNorm(4, H),
    nn.ReLU(inplace=True),
```

```
    nn.Conv3d(H, C, 1),
)
dx = 0.1 * torch.tanh(mlp(perceived))
```

Listing 6.5: Update rule as a point-wise MLP implemented by $1 \times 1 \times 1$ convolutions.

When task channels are present, the predicted increment for the task-channel slice is forced to zero, keeping those channels constant.

6.2.4 Grid3D

`Grid3D` composes `CellState`, `Perception3D`, and `UpdateRule` into a complete simulator. A single step applies a residual update and then clears voxels that are not alive after the update. Listing 6.6 summarises the logic.

```
pre_life = cell.update_alive_mask(state)
perceived = perception_op(state)
dx = update_rule(perceived, pre_life, state)
new_state = state + dx

post_life = cell.update_alive_mask(new_state)
new_state = new_state * post_life.float()
```

Listing 6.6: Summary of a single NCA update step (pseudocode).

The multi-step `forward()` loop runs `step()` repeatedly and clamps the state to `[-1, 1]` after each update. For long training unrolls, steps can optionally use gradient checkpointing via `torch.utils.checkpoint.checkpoint(...)` to reduce memory usage at the cost of recomputation during backpropagation.

The module also provides `seed_center()`, which creates an initial state with a single active voxel placed at the lattice centre. Visible channels are initialised to random colour with alpha set to 1.0, while hidden channels start at zero.

6.2.5 NCAModel

`NCAModel` provides a single entry point for scripts and notebooks. It wraps `Grid3D` and keeps the user-facing configuration in one place.

Checkpoints store a configuration snapshot alongside the learned `state_dict`. Listing 6.7 shows the structure used by `NCAModel.save()`.

```
torch.save(
    {
        "version": 1,
        "config": dataclasses.asdict(model.config),
        "state_dict": model.state_dict(),
    },
    path,
)
```

Listing 6.7: Checkpoint structure used by `NCAModel.save()`.

Figure 6.1 shows the full layer sequence in the update MLP together with the stochastic fire mask and the bounded output.

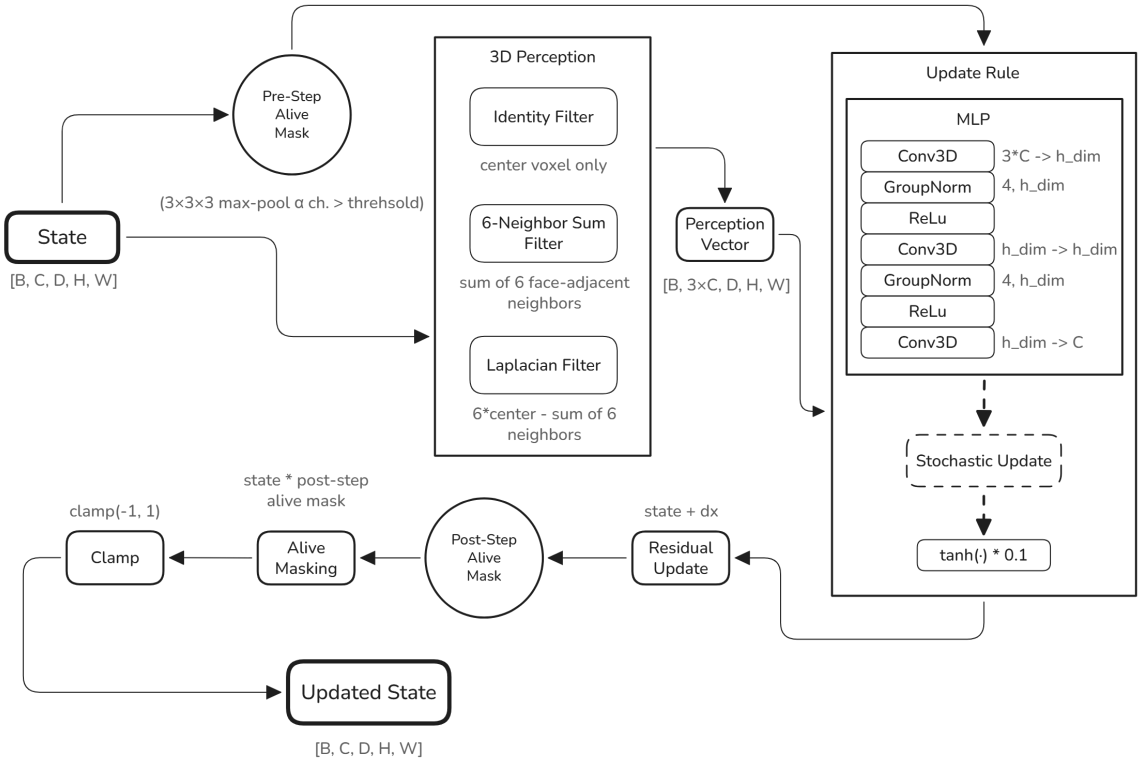

Figure 6.1: Detailed data flow through a single NCA update step, showing the full layer sequence within the update MLP, the optional stochastic fire mask, and the bounded tanh output

6.3 Training Pipeline

The conceptual structure of pool-based training, curriculum stepping, and schedule-driven parameter updates is described in Section 5.4. This section focuses on the code-level composition and the data formats used at the training boundary.

Training backends are defined by the abstract `NCARunner` interface in `src/core/runners/base.py`. The default implementation, `MorphRunner`, owns the model instance (`Grid3D`), optimiser, current target tensor, and a pool of intermediate states used as training samples. The main entry point is the generator method `train()`, which yields per-epoch metrics and makes it easy for the server to stream progress without blocking on long GPU workloads.

The second core backend, `RegenRunner` (`src/core/runners/regeneration.py`), reuses the same optimisation path but injects random local damage into pooled states before updates, providing a regeneration-oriented experiment without changing server or protocol code. The damage behaviour is controlled by `training.regen_damage_prob` and `training.regen_damage_size`.

Runner initialisation accepts voxel targets in external channels-last format `[D, H, W, C]`, which is convenient for Blender and for protocol messages. Before training, the array is converted to a channels-first tensor `[B, C, D, H, W]`. Listing 6.8 shows the conversion.

```
# NumPy: [D, H, W, C] -> torch: [B, C, D, H, W]
x = torch.from_numpy(np_array).permute(3, 0, 1, 2).unsqueeze(0).contiguous()
```

Listing 6.8: Transposition at the training boundary between channels-last voxel arrays and channels-first tensors.

Two design choices shape the training loop:

- **Pool-based training**: instead of always starting from a clean seed, training samples are drawn from a pool of previously generated states. This exposes the model to different partial solutions and stabilises long-horizon behaviour.
- **Step-count curriculum**: each epoch unrolls the automaton for a random number of steps. The allowed range is increased over time, so early training focuses on short, stable dynamics and later training learns longer growth trajectories.

Visible channels are supervised directly against the target, while hidden channels remain unsupervised. The total loss is a weighted sum of three terms: an alpha/occupancy term, an RGB colour term (when colour channels are present), and an overflow penalty that discourages activity outside the intended volume.

Listing 6.9 summarises one epoch at the level relevant for understanding the training behaviour.

```
batch = sample(pool, batch_size)
steps = randint(step_min(epoch), step_max(epoch))

pred = model(batch, steps=steps)
loss = (
    w_alpha * loss_alpha(pred, target)
    + w_color * loss_color(pred, target)
    + w_overflow * loss_overflow(pred)
)

loss.backward()
```

```
optimizer.step()
pool.update(batch, pred)
```

Listing 6.9: One training epoch in the pool-based runner (pseudocode).

Task channels and multi-target conditioning remain available at the model level, but they are not used by the two core runners. In this codebase they are demonstrated in the notebook-level metamorphogenesis experiment, where a custom runner manipulates channel slices before broadcasting snapshots. Baseline morphogenesis and regeneration both keep task channels disabled. Schedule events (learning rate, batch size, loss weights, target changes) are applied through the thread-safe `Schedule` class.

6.4 Server

The server architecture and session lifecycle are described in Section 5.5, including the message sequence (Figure 5.4) and the protocol state machine (Figure 5.5). This section focuses on the code structure of the TCP server and the background training thread.

Training sessions start with an `init` message that carries a configuration dictionary and the voxel target. Listing 6.10 shows the top-level structure used across the Blender client, server, and core runner.

```
config = {
    "cell": { ... },
    "perception": { ... },
    "update": { ... },
    "grid": { ... },
    "training": { ... },
}
```

Listing 6.10: Top-level configuration structure exchanged between Blender and the training server.

On the server side, this dictionary is used to construct the configuration objects consumed by the core model and runner. Listing 6.11 shows the mapping in a compact form.

```
cell_cfg = CellConfig(**config["cell"])
perc_cfg = PerceptionConfig(**config["perception"])
upd_cfg = UpdateConfig(**config["update"])
grid_cfg = GridConfig(**config["grid"])
```

Listing 6.11: Mapping a configuration dictionary to the component configuration objects.

The interactive training server is implemented in `src/server/`. `NCAServer` handles the TCP socket lifecycle and dispatches incoming messages. Expensive work (training and inference) is delegated to `NCATrainer`, which runs a background thread and exposes a small control API: `pause`, `resume`, `stop`, and `update_schedule`.

During training, the trainer drives the selected `NCARunner.train()` generator and broadcasts state snapshots to the client with rate limiting, so viewport updates remain smooth even when the training loop produces updates faster than the network link can handle. During inference, the trainer loads a checkpoint from raw bytes and runs forward steps without checkpointing.

Listing 6.12 shows the message dispatch loop in simplified form.

```
while True:
    msg = recv_msg(sock)
    t = msg.get("type")

    if t == "init":
        trainer.init(config, target, send_fn)
    elif t == "run_model":
        trainer.run_inference(model_bytes, phase_steps, broadcast_every, send_fn)
        # phase_steps in the wire message is used as total forward steps.
    elif t == "pause":
        trainer.pause()
    elif t == "resume":
        trainer.resume()
    elif t == "stop":
        trainer.stop()
    elif t == "update_schedule":
        trainer.update_schedule(events)
```

Listing 6.12: Server-side message dispatch loop (simplified).

The trainer API is intentionally small and is designed to be used directly from the message handlers. Listing 6.13 summarises the typical call patterns.

```
trainer = NCATrainer(base_dir="runs", checkpoint_interval=500)

# Start/replace a training run.
trainer.init(config, target, send_fn)

# Training control.
trainer.update_schedule(events)
trainer.pause()
trainer.resume()
trainer.stop()

# Inference from a raw checkpoint sent by the client.
trainer.run_inference(model_bytes, phase_steps, broadcast_every, send_fn)

# Optional: inject a custom backend that implements NCARunner.
trainer = NCATrainer(runner_factory=MyCustomRunner)
```

Listing 6.13: Typical trainer usage patterns from the server layer (pseudocode).

For notebook/Colab experiments with Blender visualization, the minimal pattern is to define a small `NCARunner` subclass in a notebook cell, inject it via `runner_factory`, and start the server. Listing 6.14 shows this setup in compact form.

```
from src.core.runners import NCARunner, MorphRunner
from src.server.trainer import NCATrainer
from src.server.server import NCAServer

class NotebookRunner(NCARunner):
    def __init__(self):
        self.base = MorphRunner(verbose=False)
    def init(self, config, target): self.base.init(config, target)
    def train(self, schedule=None): yield from self.base.train(schedule=schedule)
```

```
    def snapshot(self): return self.base.snapshot()
    def set_target(self, target): self.base.set_target(target)
    def on_event(self, event): return self.base.on_event(event)

trainer = NCATrainer(runner_factory=NotebookRunner, verbose=False)
NCAServer(host="0.0.0.0", port=5555, trainer=trainer).start()
```

Listing 6.14: Minimal notebook/Colab custom runner injection (simplified).

6.4.1 Protocol

Protocol behaviour at the session level is described in Section 5.5. This subsection summarises the concrete framing and payload encoding used in the implementation.

Each message consists of a four-byte big-endian unsigned integer carrying the JSON payload length, followed by the UTF-8 JSON itself (Figure 6.2).

Tensor payloads (targets and broadcast states) are transmitted as base64-encoded float32 byte arrays together with an explicit `shape` field. Model checkpoints are transmitted as base64-encoded raw `.pt` bytes during inference requests. Figure 6.3 summarises the message types used by the client and the server.

At protocol level, message types define control behaviour, while payload fields can be adapted where needed. In practice, `init` is the primary extension point: researchers may add, remove, or reshape fields (for example to support multi-target voxelisation or different configuration structures). Control messages such as `pause`, `resume`, and `stop` are intentionally stable, because their behaviour is transport-level session control and should stay predictable across clients.

When payload semantics are changed across tools, introducing protocol versioning is recommended to keep compatibility explicit.

The protocol layer itself is intentionally layout-agnostic: it transports typed payload bytes together with explicit shape metadata, but does not define a single visualisation axis convention. In the current implementation, the server broadcasts visible-channel state slices and the Blender side performs the final conversion to display voxel layout. This separation is deliberate, because experiments can change internal state organisation while keeping the wire framing stable and updating only the conversion adapters.

Listing 6.15 shows the core framing operation used by both the client and server.

```
payload = json.dumps(msg).encode("utf-8")
wire = struct.pack(">I", len(payload)) + payload
sock.sendall(wire)
```

Listing 6.15: Length-prefixed framing for TCP messages.

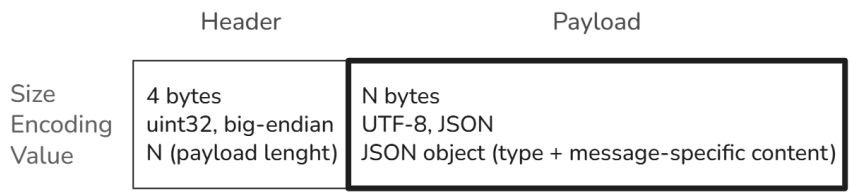

Figure 6.2: Wire message format. Each message consists of a four-byte big-endian unsigned integer carrying the payload byte length, followed by the UTF-8 JSON payload

Message Type	JSON Fields
init	type, config{}, target (base64), target_shape[]
run_model	type, model_b64, phase_steps, broadcast_every
update_schedule	type, events[]
stop	type
pause	type
resume	type
state	type, data (base64), shape[], epoch, loss
ack	type, message
error	type, message

Figure 6.3: Protocol message types and their JSON fields. Messages above the divider are sent by the client; messages below are sent by the server

6.4.2 Logging

The role of logging and stored artefacts is described in Section 5.5. In the implementation, run outputs are written by `NCALogger` (`src/server/logger.py`). Each training run claims the next available `runs/run_NNN/` directory and persists both metadata and artefacts:

- `meta.json`: configuration snapshot, timestamp, and optional git hash,
- `loss.csv`: per-epoch loss components,
- `events.jsonl`: applied schedule events (one JSON record per line),
- `checkpoints/`: periodic `.pt` checkpoints containing `config` and `state_dict`.

Listing 6.16 shows the on-disk layout created for each run.

```
runs/
    run_003/
        meta.json
        loss.csv
        events.jsonl
        checkpoints/
            model_ep00500.pt
            model_ep01000.pt
            ...
```

Listing 6.16: Run directory layout produced by the training server.

6.5 Blender Add-on

The add-on architecture (UI panels, operators, and the data flow to the server) is described in Section 5.6. This section highlights the implementation structure and the Blender-specific threading pattern needed for interactive visualisation.

The add-on is implemented in the `blender/` package. UI properties are stored in scene-level property groups and exposed through sidebar panels. Actions are implemented as Blender operators that connect to the server and request either training or inference.

Voxelisation and viewport rendering are handled by `voxel_utils.py`. Meshes are converted into dense voxel arrays for transmission and then reconstructed into a voxel mesh for display.

Network communication is kept off the Blender main thread. `NCAClient` runs a listener thread that receives protocol messages and decodes broadcast states. The listener stores only the latest received state into a shared buffer and schedules a timer callback. The timer runs on the main thread and performs the actual scene updates, which avoids thread-safety issues in Blender.

Listing 6.17 shows the buffer-and-timer pattern used for state visualisation.

```
def on_state(arr):
    global pending
    pending = arr
    bpy.app.timers.register(poll_state, first_interval=0.05)

def poll_state():
    global pending
    if pending is not None:
        render_state(pending)
        pending = None
    return 0.1
```

Listing 6.17: State visualisation via a listener thread and a main-thread timer (simplified).

Chapter 7

Testing

Testing combined lightweight automated checks with exploratory evaluation in notebooks and interactive end-to-end runs through Blender. The automated test suite is intentionally small and focuses on core tensor invariants, runner and trainer API behaviour, and client/server command flow rather than comprehensive coverage.

Automated tests are implemented using `pytest` in the `tests/` directory:

- `test_core.py` Unit tests for the core model components. The suite verifies the alive-mask computation in `CellState`, the fixed perception kernels and frozen weights in `Perception3D`, basic invariants of `UpdateRule` (zero-initialised final layer and masking of task-channel increments), and seed initialisation in `Grid3D` including the optional task-channel one-hot slice. It also checks configuration and checkpoint round-trips for `NCAModel`.

- `test_protocol_schedule.py` Currently contains additional runner-facing contract checks (snapshot shape and event handling) that overlap with `test_runner_validation.py`. It is used as a lightweight regression file for training-interface behaviour after the runner abstraction changes.

- `test_runner_validation.py` Focused tests for `MorphRunner` and `RegenRunner`: initialisation, one-epoch training metrics, snapshot contract, event handling through `on_event()`, abstract-interface compliance, and input-shape validation.

- `test_server_client.py` Integration-style checks for server–client interaction at the API boundary. The Blender client is exercised with a fake socket and monkeypatched send function to confirm the generated control messages. The server dispatch loop is tested against a fake trainer to confirm message routing. Trainer broadcast rate limiting is also verified.

Automated tests are complemented by practical protocol verification using Wireshark. This was used to monitor the TCP traffic between the Blender client and the server, inspect the framed JSON messages on the wire, and validate message framing and broadcast cadence during interactive sessions.

During early development, Jupyter notebooks were used as an exploratory testbed for the core model and training behaviour. These notebooks rely on the lightweight `src/viz/` and `src/io/` helpers, which is the primary reason these modules exist despite not being part of the main core–server–Blender workflow. Each notebook focuses on a different aspect:

- `01_basic_functionality.ipynb`: stepping the untrained model forward and inspecting basic state statistics and alive voxels.
- `02_sphere_training_alpha.ipynb`: an analytic voxel target (sphere) and alpha-only optimisation with slice and volume visualisations.
- `03_donut_training_alpha.ipynb`: a mesh-based target imported from an OBJ file and trained in an occupancy-only setting.
- `04_donut_training_rgba.ipynb`: an RGBA target and colour-supervised training with volume visualisation.

After the Blender add-on was implemented, it became the primary environment for end-to-end testing and qualitative observation of training dynamics, target voxelisation, protocol responsiveness, and viewport rendering. The resulting experiments and observations are discussed in the next chapter.

Chapter 8

Experiments

TODO

Chapter 9

Conclusion

TODO

Bibliography

[1] Berlekamp, E. R.; Conway, J. H. and Guy, R. K. *Winning Ways for Your Mathematical Plays, Volume 2*. Academic Press, 1982.

[2] Chan, B. W.-C. *Lenia: Biology of Artificial Life*. 2019.

[3] Community, P. *PyTorch Documentation*. 2025. Available at: https://docs.pytorch.org/stable/index.html.

[4] Cook, M. *Universality in Elementary Cellular Automata*. 2004.

[5] Eddy, W. *Transmission Control Protocol (TCP)* RFC 9293. 2022. Available at: https://doi.org/10.17487/RFC9293.

[6] Gardner, M. *Mathematical Games: The fantastic combinations of John Conway's new solitaire game "Life"*. 1970. Available at: https://web.stanford.edu/class/sts145/Library/life.pdf.

[7] Goodfellow, I.; Bengio, Y. and Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[8] Horváth, D. and Levin, M. *Self-Classifying MNCA Distill*. 2021. Available at: https://distill.pub/2021/self-classifying-mnca/.

[9] Ilachinski, A. *Cellular Automata: A Discrete Universe*. World Scientific, 2001. ISBN 981-02-4623-4.

[10] Jha, A. *3D Growing Neural Cellular Automata*. GitHub, 2024. Available at: https://github.com/Aadityaza/3d-Growing-neural-cellular-automata. Originally published under username Aadityaza.

[11] Kinderen, E. de and Wuensche, A. 3D CA in a fluid dynamics PDE. In: *Proceedings of the Second International Conference on Evolvable Systems: From Biology to Hardware*. Springer, 1998.

[12] Laine, S. and Karras, T. Efficient sparse voxel octrees. In: *Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. New York, NY, USA: Association for Computing Machinery, 2010. ISBN 9781605589398. Available at: https://doi.org/10.1145/1730804.1730814.

[13] Marinescu, D. C. Chapter 2 - Nature-Inspired Algorithms and Systems. In: Marinescu, D. C., ed. *Complex Systems and Clouds*. Boston: Elsevier, 2017, p. 33–63. Computer Science Reviews and Trends. ISBN 978-0-12-804041-6. Available at: https://www.sciencedirect.com/science/article/pii/B9780128040416000025.

[14] Mordvintsev, A.; Randazzo, E.; Niklasson, E. and Levin, M. *Growing Neural Cellular Automata*. 2020. Available at: https://distill.pub/2020/growing-ca/.

[15] Museth, K. VDB: High-resolution sparse volumes with dynamic topology. *ACM Trans. Graph.* New York, NY, USA: Association for Computing Machinery, 2013. ISSN 0730-0301. Available at: https://doi.org/10.1145/2487228.2487235.

[16] Neumann, J. von. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.

[17] Niklasson, E.; Mordvintsev, A.; Randazzo, E. and Levin, M. Self-Organising Textures. *Distill*, 2021. Https://distill.pub/selforg/2021/textures/.

[18] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Available at: https://arxiv.org/abs/1912.01703.

[19] Rafler, S. *Generalization of Conway's "Game of Life" to a continuous domain – SmoothLife* arXiv preprint arXiv:1111.1567. 2011. Available at: https://doi.org/10.48550/arXiv.1111.1567.

[20] Randazzo, E.; Mordvintsev, A. and Levin, M. *Persistent Memory in Growing Neural Cellular Automata* Google Research Blog. 2021. Available at: https://ai.googleblog.com/2021/12/persistent-memory-in-neural-cellular.html.

[21] Rossi, F. and Orsini, M. *3D Growing Neural Cellular Automaton*. GitHub, 2021. Available at: https://github.com/SkyLionx/3d-cellular-automaton. Originally published under username SkyLionx.

[22] Schulzrinne, H.; Casner, S.; Frederick, R. and Jacobson, V. *RTP: A Transport Protocol for Real-Time Applications* RFC 3550. 2003. Available at: https://doi.org/10.17487/RFC3550.

[23] Schulzrinne, H.; Rao, A.; Lanphier, R.; Westerlund, M. and Stiemerling, M. *Real Time Streaming Protocol (RTSP) 2.0* RFC 7826. 2016. Available at: https://doi.org/10.17487/RFC7826.

[24] Sudhakaran, S.; Grbic, D.; Li, S.; Katona, A.; Najarro, E. et al. *Growing 3D Artefacts and Functional Machines with Neural Cellular Automata*. July 2021, ALIFE 2021: The 2021 Conference on Artificial Life, p. 108. ALIFE 2022: The 2022 Conference on Artificial Life. Available at: https://doi.org/10.1162/isal_a_00451.

[25] Tobias, P. A. G. *3D Cellular Automata*. 2009. Available at: https://eprints.bournemouth.ac.uk/17408/.

[26] Turing, A. M. The Chemical Basis of Morphogenesis. *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences*, 1952, vol. 237, no. 641, p. 37–72.

[27] Wolfram, S. *Statistical Mechanics of Cellular Automata*. 1983.

[28] Wolfram, S. *A New Kind of Science*. Wolfram Media, 2002. ISBN 1-57955-008-8. Available at: https://www.wolframscience.com/nks/.